

☒ Loveable Cheat Sheet.



📑 1. Project Knowledge Base (Think of This Like Your AI's Brain)

What: Give Loveable some background *before* asking it to build things. Just type out a short high-level summary so the AI understands what it's working on.

Why it matters: If the AI doesn't know what the project is about or what tools you're using, it will guess. And that's when weird or useless code starts happening.

🧠 What to Include Before You Start Prompting:

Just paste this kind of info into the setup or first message:

- **Project Overview** (1–3 lines):
"This is a web app for freelancers to track invoices and payments. It has a dashboard, user logins, and PDF export features."
- **What You're Using to Build It** (don't overthink it):
*"Use React, Tailwind CSS, Supabase, and ShadCN for UI components."
(If you don't know, just describe what tech you want to use — like "a modern website with login, some charts, and mobile support." The AI can suggest tech too.)*

- **What's Already Built (if anything):**

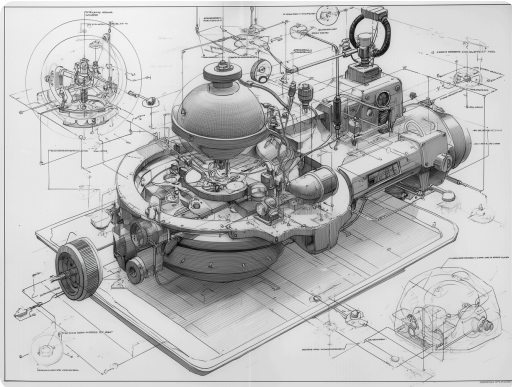
"I've already got login working and a basic dashboard layout. Just need help building the invoice page next."

- **What You Want Help With First:**

"Start by building the invoice creation form. It should include fields for client name, line items, and tax."

Pro Tip: You can write this in plain language — it's not school. You're basically saying:

🧑 "Here's what I'm building, what I'm using, and what I want help with next."



👉 2. Prompt Engineering via the CLEAR Framework (Expanded)

Context → Set the stage (e.g. "You're a senior frontend dev using shadcn and Tailwind")

Location → Name exact file/component/folder

End Goal → Describe what you want built ("Build pricing card component with toggle")

Approach → Methodology, tech choices ("Use React + Tailwind, functional components only")

Restrictions → Guardrails ("Don't touch auth logic or base layout file")

Bad: "Make it prettier"

Great: "Improve button hover animations using Tailwind transition utilities. Keep it accessible."



🧱 3. Architecture-First Mentality

Start at the *scaffold* level. Force AI to build a mental model of your structure before diving into features.

Steps:

1. Define layout & routing first
2. Then global components (Navbar, Layout, Footer)
3. Then screens (Dashboard, Profile)
4. Then nested components (Card, Modal, Form)

Include Early:

- Routing strategy (e.g. App Router in Next.js 14)
- Folder structure preferences
- Global styles/themes



4. Responsive Design = Mandatory

Rule: Always say “make it mobile-first and responsive on all breakpoints”

Why: Otherwise you’ll get a fixed-width desktop design

How:

- Explicitly mention Tailwind’s built-in breakpoints (sm, md, lg, xl, 2xl)
- Use shadcn’s responsive primitives when applicable
- Avoid custom media queries unless absolutely necessary

Pro Tip: Reference your Figma or Excalidraw if available.

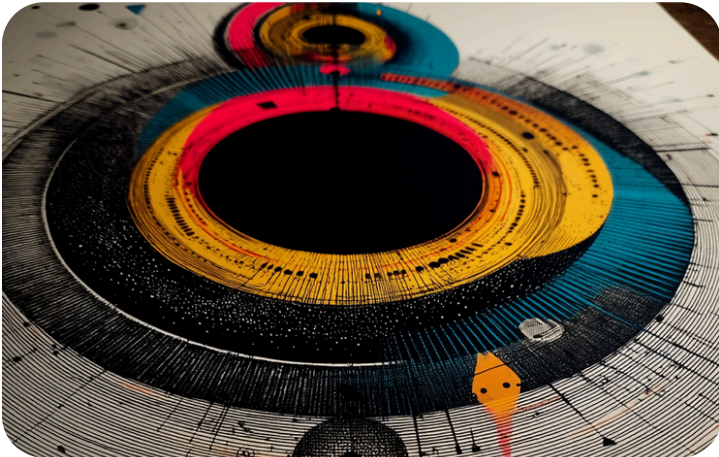


🎯 5. Precise Targeting = No Accidental Code Overwrites

Problem: AI loves to rewrite unrelated files.

Solution:

- Use **"Select"** in Loveable or say:
→ "Only edit components/PricingCard.tsx and nothing else"
- Lock files that should remain untouched
- Reiterate scope boundaries in every follow-up prompt



🔄 6. Incremental Development: Token Efficiency & Accuracy

Avoid: "Build the whole dashboard, auth flow, and settings panel"

Do:

- One component per prompt
- One logic update per request
- One test case per function

Split into 3 tracks:

1. UI
2. State management
3. API/data logic

This reduces hallucinations and token bloat.



👁 7. Visual Context › Written Descriptions

Include:

- Figma/Excalidraw links
- Screenshots (annotated if possible)
- Style references (e.g. “Make this like Stripe’s dashboard but warmer colors”)

Use Mermaid or Freeform for architecture, DB schema, or flow diagrams.



🎨 8. Vibe: Define the Mood & Aesthetic

More powerful than most realize. Treat your prompt like you're directing a designer:

Examples:

- “Professional, techy, Stripe-like feel”
- “Playful, friendly, using rounded corners and pastel palette”
- “Serious, clean, legal SaaS dashboard”

Set the tone before writing a single line of code.



🍷 9. Error Handling and Debugging Etiquette

Bad: “It doesn’t work”

Better: Paste exact error, say when it happens, and what you tried

Best:

- Add `console.log`, `try/catch` or debugger before asking Loveable
- Ask for *reasoning first*, code second:
 - “Can you analyze why this fetch call times out?”
 - “Now apply a fix, but keep the retry logic intact”

Use Chat Mode: When you want thinking, not just code output.

✅ Summary Cheatsheet

Area	Key Practice
Project Setup	Create detailed knowledge base file
Prompting	Use CLEAR (Context, Location, End Goal, Approach, Restrictions)
Architecture	Start from framework and folder layout
Responsiveness	Always mention mobile-first + Tailwind breakpoints
Scope Control	Explicitly limit edits to named files/components
Development Strategy	Incremental (one feature, one change)
Visual Aids	Include screenshots, diagrams, moodboards
Tone + Aesthetic	Define “vibe” like a creative director
Debugging	Share full error + logs + file context
File Locking	Prevent unintentional edits
Mode Choice	Chat for planning/debugging, Default for quick edits